Matej Dedina

A C Data Structure Book

BOOK I

April 20, 2026

Springer Nature

Preface

This book describes the process of designing and implementing a personal data structure library in the C programming language. Each implementation is centered around two types of implementations - an infinitely expandable structure able to hold any amount of elements, and a finite one able to hold only a certain maximum amount.

The goal of this work is not to be the go-to data structure book for C, on the contrary, the name is meant to represent an affordable book for the public which doesn't take itself seriously.

The book will be divided into parts classifying all structures into categories of chapters. Each chapter will explain the process of creating said structure and the code used to make it.

Since the code is in C a decent understanding of the programming language is required, that primarily includes - structures (struct), pointers, memory allocation, and last but certainly not least - function pointers.

All the source code will be available at github under the `Unlicense License`.

If I have to put a disclaimer I just want to clarify that the reason why some paragraphs may be written like this is because the book was created using LaTeXin Visual Studio Code and I hate seeing `Overfull \hbox` highlights.

Contents

1 The cerpec data structure library 1
1.1 The main header 1
1.1.1 Growth factor and chunk size 1
1.1.2 Error and validation handling 2
1.1.3 Custom memory allocator 3
1.1.4 Function pointers 4
1.2 Implementation philosophies 5
1.2.1 Mantras 6
1.2.2 How others did it 6
1.2.3 Python 6
1.2.4 Haskell 7
Further reading 7

I Restricted sequential data structures

2 Stacks 11
2.1 Implementations of a Stack 11
2.1.1 The humble linked list 13
2.1.2 The prideful array 14
2.1.3 Singly linked list of array elements 15
2.2 Array stack structure code 15
2.2.1 The infinite stack structure 16
2.2.2 Shrinking, expanding, resizing 16
2.2.3 Creating the stack 16
2.2.4 The makings of a stack 17
2.2.5 Stack destruction 17
2.2.6 Clearing up the stack 19
2.2.7 The deep/shallow copy 19
2.2.8 Checking emptiness 20
2.2.9 Pushups to the top 21
2.2.10 Popping it 22
2.2.11 Reaching the peep 23

2.2.12 For each element . . . 23
2.2.13 Applying mostly sorting . . . 24
2.3 Example usage . . . 24
2.3.1 Creating and clearing/destroying a stack of integers . . . 24
2.3.2 Pushing, peeping and popping integer elements . . . 25
2.3.3 Iterating and sorting . . . 26

3 Queue . . . 27
3.1 Implementations of a queue . . . 27
3.1.1 The free linked list . . . 29
3.1.2 The confined array . . . 29
3.1.3 Tail-based circular linked list of arrays . . . 30
3.2 Tail-based circular linked list of array code . . . 31
3.2.1 The infinite queue structure . . . 31
3.2.2 Creating the queue . . . 33
3.2.3 The makings of a queue . . . 33
3.2.4 Queue destruction . . . 34
3.2.5 Clearing up the queue . . . 35
3.2.6 The deep/shallow copy . . . 36
3.2.7 Checking emptiness . . . 38
3.2.8 Enqueueing it to the back . . . 38
3.2.9 Dequeueing it . . . 40
3.2.10 Peeking the front . . . 41
3.2.11 For each element . . . 41
3.2.12 Apply sorting or something . . . 42
3.3 Example usage . . . 44
3.3.1 Creating and clearing/destroying a queue of integers . . . 44
3.3.2 Enqueueing, peeking and dequeueing integer elements . . . 45
3.3.3 Iterating and sorting . . . 45
Further reading . . . 46

4 Deque . . . 47

II Linked lists

Chapter 1
The cerpec data structure library

The name `cerpec` may look familiar if you're a Polish speaker, but the word actually comes from the Eastern Slovak word for suffering. One can also see the similarities when writing and reading the standard Slovak, Eastern Slovak, and Polish words next to each other, like this `trpieť - cerpec - cierpieć`.

The reasons for choosing this name are three-fold; making this library was like suffering through psychological pain; Eastern Slovak infinitives ending with -c (standard Slovak with -ť) go nicely with the `C` in the C programming language; and lastly,I doubt anyone would use `cerpec` for trademarking reasons since it's a dialectic word.

1.1 The main header

The `cerpec.h` serves as the parent containing all the necessary definitions shared among all the data structures. The header can be divided into four parts.

1.1.1 Growth factor and chunk size

The growth factor `CERPEC_FACTOR` represents the factor by which to determine the previous and next memory size to resize into when we're either out of space or if smaller space is available.

include/cerpec.h

```
#define CERPEC_FACTOR 2
```

The chunk size `CERPEC_CHUNK` is used to avoid a small starting size where the program might be too busy just reallocating the structures instead of performing operations, graphically exemplified in Figure 1.1. Basically, instead of starting at length 1 after inserting the first element, it starts at a specific higher power of two, in the case of this project - 256, and grows exponentially as in Listing 1.1.1.

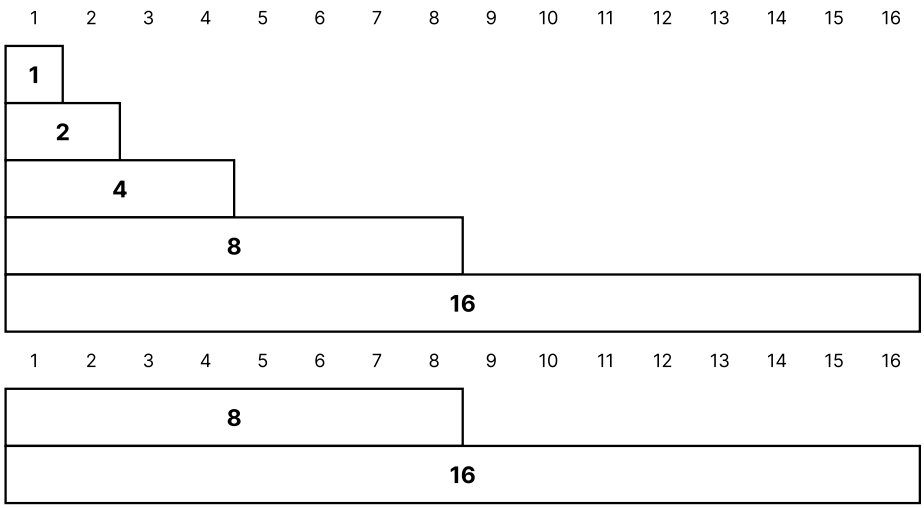

Fig. 1.1 Example growth if chunk size is 1 (top) vs 8 (bottom). The height of the pyramid represents the number of memory growths performed.

include/cerpec.h

```
// define chunk size to expand and contract all data structures
#if !defined(CERPEC_CHUNK)
#   define CERPEC_CHUNK 256
#elif CERPEC_CHUNK ≤ 0
#   error "Chunk size must be greater than zero."
#elif (CERPEC_CHUNK & (CERPEC_CHUNK - 1))
#   error "Chunk size must be a power of 2."
#endif
```

1.1.2 Error and validation handling

Assertions are a simple way to check if certain properties are true either before or after an action has been performed. For `cerpec` there are two types of assertions - `error` and `valid` (see Listing 1.1.2).

The `error` assertion errors when something bad that shouldn't happen happens, common examples include - a parameter being NULL, memory allocation failing, no element being found. The `valid` assertion validates the state of the structure, for example if `length` member is less than or equal to `capacity` member, if element `size` member is greater than zero. `valid` assertions are just less important conditions that should be disabled in production for better performance.

It's just a means to allow the user to specify which assertions they want to check and disable. The way to disable `valid`, `error` and all assertions altogether are via

defining `NVALID`, `NERROR` and `NDEBUG` respectively, either through compiler flag macro definitions or using `#define` prior to any data structure inclusion. I believe that in the future more assertions will be added.

include/cerpec.h

```
#ifdef NDEBUG
#   define NVALID
#   define NERROR
#endif

#ifndef NVALID
#   define valid(condition) assert(condition)
#else
#   define valid(condition) (void)(0)
#endif

#ifndef NERROR
#   define error(condition) assert(condition)
#else
#   define error(condition) (void)(0)
#endif
```

1.1.3 Custom memory allocator

Each structure has a pointer to a custom memory allocator which can be used to allocate, free and reallocate memory based on arguments. The `standard` memory allocator is a constant external variable that uses the standard library's `malloc`, `free` and `realloc` functions in the `create_*` data structure functions. Below is an example of how the book shows standard C library functions, these ones taken from the Linux manual pages.

Linux manual page - malloc(3)

The `malloc()` function allocates size bytes and returns a pointer to the allocated memory. The memory is not initialized. If size is 0, then `malloc()` returns a unique pointer value that can later be successfully passed to `free()`.

Linux manual page - free(3)

The `free()` function frees the memory space pointed to by p, which must have been returned by a previous call to `malloc()` or related functions. Otherwise, or if p has already been freed, undefined behavior occurs. If p is NULL, no operation is performed.

Linux manual page - realloc(3)

The `realloc()` function changes the size of the memory block pointed to by p to size bytes. The contents of the memory will be unchanged in the range from the start of the region up to the minimum of the old and new sizes. If the new size is larger than the old size, the added memory will not be initialized.
If p is NULL, then the call is equivalent to `malloc(size)`, for all values of size.
If size is equal to zero, and p is not NULL, then the call is equivalent to `free(p)`.
Unless p is NULL, it must have been returned by an earlier call to malloc or related functions. If the area pointed to was moved, a free(p) is done.

The only way to use custom memory is through defining three custom function pointers and putting them as parameters for the `compose_memory` builder, as is the case in Listing 1.1.3. Each data structure has a function starting with `make_` which allows more customization. Right now it only allows to add a custom allocator, but in the future it might allow more parameters for customization.

include/cerpec.h

```
typedef void * (*alloc_fn)   (size_t const, void *);
typedef void * (*realloc_fn) (void *, size_t const, void *);
typedef void   (*free_fn)    (void *, void *);

typedef struct memory {
    void * arguments;
    alloc_fn alloc;
    realloc_fn realloc;
    free_fn free;
} memory_s;

memory_s compose_memory(alloc_fn const alloc, realloc_fn const realloc,
     free_fn const free, void * const arguments);
```

The standard memory allocations can be accessed through the constant external variable `standard`.

include/cerpec.h

```
extern const memory_s standard;
```

1.1.4 Function pointers

Listing 1.1.4 is a code snippet list of all available function pointers which allow for more generic and user-defined data manipulation.

include/cerpec.h

```c
typedef void   (*set_fn)     (void * const element);
typedef void * (*copy_fn)    (void * const destination, void const * const
    source);
typedef size_t (*hash_fn)    (void const * const element);
typedef int    (*compare_fn) (void const * const a, void const * const b);
typedef bool   (*filter_fn)  (void const * const element);
typedef bool   (*handle_fn)  (void * const element, void * const arguments)
    ;
typedef void   (*process_fn) (void * const array, size_t const length, void
     * const arguments);
typedef void   (*operate_fn) (void * const result, void const * const a,
    void const * const b);
```

The explanation for each of the function pointers is as follows:

1. `set_fn` - a function pointer that allows to manipulate a generic element pointer directly, can be used to set the specified element to zero, deallocate memory, increment and decrement values, etc.
2. `copy_fn` - a function pointer to create a copy of a specific element. It is possible to also copy all nested sub-elements either deeply or referenced shallowly, starting from a source element reference into a destination one.
3. `hash_fn` - a function pointer which takes in a generic element and returns a `size_t` value representing a hash. This hash can be used to index the value into a hash array.
4. `compare_fn` - a function pointer that compares elements `a` and `b`, and returns 0 if equal, negative if `a` is smaller, and positive otherwise.
5. `filter_fn` - a function pointer that returns `true` if the specified element meets a certain criteria, `false` otherwise. Can be used to extract elements in linear list structures, for example odd/even numbers, prime numbers, but also whitespace, alphanumeric characters, etc.
6. `handle_fn` - a function pointer that can handle a single element based on arguments. This function allows to iteratively change elements until `false` is returned (kinda like a `break` in a classic c loop.). The closest thing to this would be a `foreach` loop body in other languages like C#.
7. `process_fn` - a function pointer that allows to process an array of elements instead of iteratively walking through each one. Can be used to sort a sequence of element in array form for linear data structures like stacks, queues, deques and lists.
8. `operate_fn` - a function pointer that performs an operation on two elements and saves the result inside the first `result` parameter. The only usage is in the $F(n) = G(n) + H(n)$ calculation for the A* path finding algorithm ($F(n)$ being `result` and the rest being the other two parameters).

1.2 Implementation philosophies

This section simply contains my personal preferences and opinions about certain design and code structure choices. I hereby give you permission to disagree with everything written here.

1.2.1 Mantras

I made up these following mantras used by the data structures:

- Snake case.
- Public header functions end with the structure name, private source ones begin with an underscore and the name, like this public: `push_istack`, private: `_istack_resize`.
- `char *` elements - All contained element arrays must be pointers to characters. The reason is simple, the `sizeof` operator returns the number of `char` elements a type can hold. This is important for traversing the array, as getting to an element uses this formula: $element = elements_array + (index_of_element * size_of_element)$.
- East const - The code uses east const style for better readability. Since I decided to change from West const to east const while being mostly done with the library there might be some West const left. Here's west const for constant integer and constant pointer to constant integer - `const int`, `const int * const`; and east const - `int const`, `int const * const`.
- Index-relativity - Technically every array, like `element`, `next`, `prev` in lists, can be either written inside a node struct or divided into arrays where the node itself becomes an array index. Classic nodes are just indexes in the array of available memory, meanwhile dividing node members into individual arrays makes the array index relative to its data structure. Simply said, instead of having a doubly linked list with a node struct member with `element`, `next` and `prev` members, we have just the list structure with an `elements` array, `nexts` array, and `prevs` array member and the 'node' is just an index value.

1.2.2 How others did it

1.2.2.1 C++

The C++ programming language and its standard data structures have been a great source of inspiration for this project. Many structures follow the same design and implementations as the C++ standard library. Every time I wasn't sure how to implement something I looked up how C++ did it and adapted it to C. With that said, there were some gaps.

1.2.3 Python

From Python I took the idea of set operations. I decided to implement them just like Monty Python intended. Jokes aside, I find the Python set operations way more human than what C++ offers.

1.2.4 Haskell

Haskell is probably the most quintessential programming language for my library. Using function pointers as parameters is the thing that defines this project and without Haskell's higher order functions serving as inspiration I wouldn't be able to make the project into reality. Haskell lists also played a significant role of inspiration. As is evident in the many operations one can perform on `cerpec's` lists.

Further reading

[1] cplusplus.com. C++ Reference. Accessed: 2026-03-16. 2026. URL: https://cplusplus.com/reference/stl/.

[2] B.W. Kernighan and D. Ritchie. C Programming Language. Pearson Education, 1988. ISBN: 9780133086218. URL: https://books.google.sk/books?id=Yi5FI5QcdmYC.

[3] Michael Kerrisk. Linux man pages online. https://man7.org. Accessed: 2026-03-16. Linux Foundation, 2026.

[4] Lewboski. Code Reliability with Assertions. www.youtube.com/watch?v=ayfw_NFBWQg&t. Accessed: 2026-03-16. Nov. 2025.

[5] NoRedInk. Module List. https://hackage.haskell.org. Accessed: 2026-03-16. 2026.

Part I

Restricted sequential data structures

The name of the first part comes from the inability to find a categoric name for only stacks, queues and deques. Online sources categorize those as linear dynamic data structures, the problem is... linked lists are also part of this category. I want to put list implementations into a separate book part, thus I hereby coin the term `Restricted sequential data structures` as a subcategory of dynamic linear data structures. I know nobody will take this declaration seriously, but for the sake of this book please bear with it.

Chapter 2
Stacks

Lets imagine we have a stack of books. The normal way we can add books to the stack is by putting them at the top, if we put a book with the cover pointing upwards one also knows what book exactly is at the top. To remove a book we just grab the top one and put it somewhere else.

By making space for another stack of books and adding all the books from the first one, while the constrains of putting and removing still remain, we get a new stack, but with a specific property that, when compared with the initial one, makes us tilt our head (literally). The new stack is just like the last one, but with the books in opposite order. This little property allows us to invert the order of any linear ordering of elements without knowing how they're implemented, with just adding and removing of elements.

Continuing with the two stack analogy, what if, instead of books we stored a game of chess. By adding these moves one-by-one (see Figure 2.1): E2-E4, E7-E5, G1-F3, B8-C6, D2-D4, E5-D4, F3-D4, F8-C5, C2-C3, D8-F6, D4-C6, F6-F2; we get probably the funniest chess game in modern history [1]. And on top of that if we remove the two moves from the top and reset the pieces (F6-F2 becomes F2-F6), iteratively it is possible to return to the initial state of the game, see Figure 2.2. And to top it all of, as probably every data structure book and article about stacks writes this mechanism is used by search engines when you want to return to the previous pages, including back and forth traversal.

Stacks are also used to store function calls in programming languages like C to be able to know to what previous in memory place the program should go to after the top function returns.

2.1 Implementations of a Stack

Every computer science student learns that the two most common ways to implement a stack in C are:

[1] that can be watched via `https://www.youtube.com/watch?v=e91M0XLX7Jw`

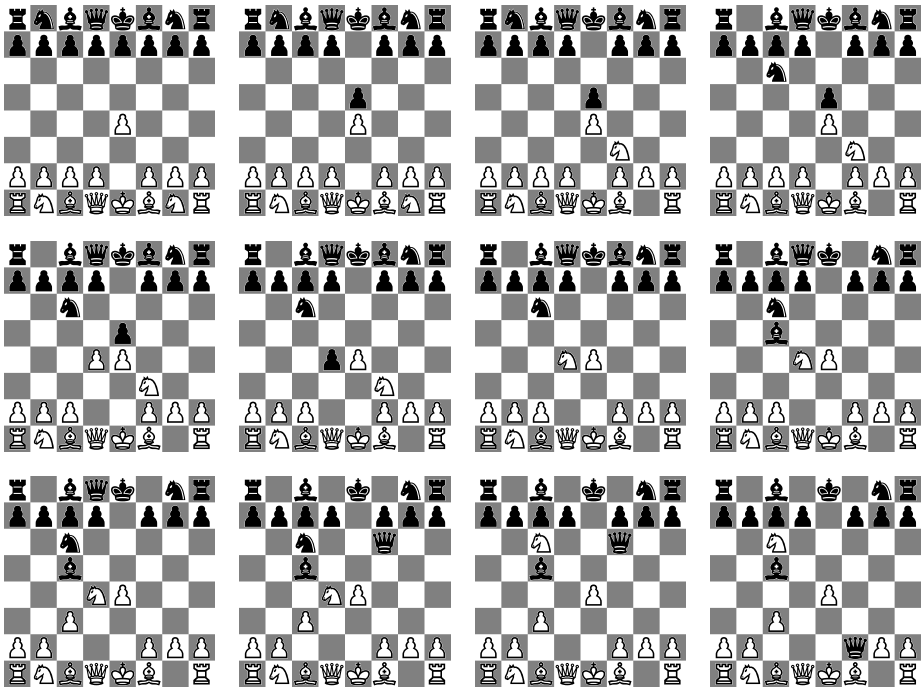

Fig. 2.1 The entire game of chess between content creators xQc and MoistCr1tikal with the latter coming on top.

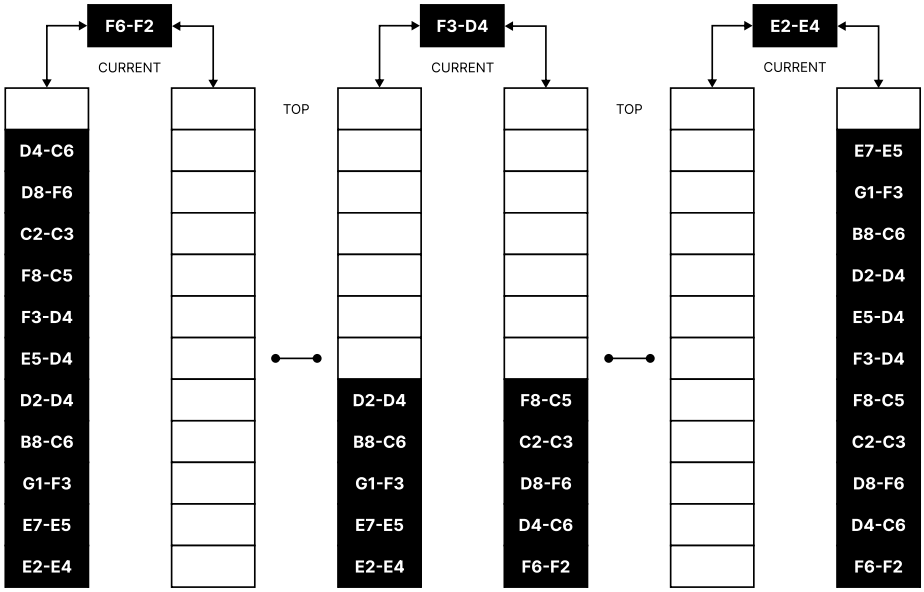

Fig. 2.2 Visual example of two stacks being used to track the chess game, (left-right) plays the game backwards, (right-left) plays the game forward

- Singly linked lists
- Static/dynamic arrays

So why does this book have three subsections for stack implementations?

2.1.1 The humble linked list

For a more in-depth explanation of lists you can go to Part II's lists, but in short a linked list is a linear data structure which allows us to store data sequentially as well as store a reference to the next data item, like in Figure 2.3.

Fig. 2.3 A simple graphical linked list example.

The linked list consists of two special nodes which represent the beginning and the end - head and tail. Most linked list implementation will store the reference to the head, so it is generally easy to access the first element as oppose to the tail, where the user needs to follow all the arrows until they reach it.

Adding, removing and accessing the first top node only requires a reference to the head, shown in Figure 2.4. Adding is just creating a new node, then making its reference point to the stack list's head, and lastly we change the head itself into the new node. Removal is storing the top node, then change stack's head to the next reference (head's next node), and destroying the removed node.

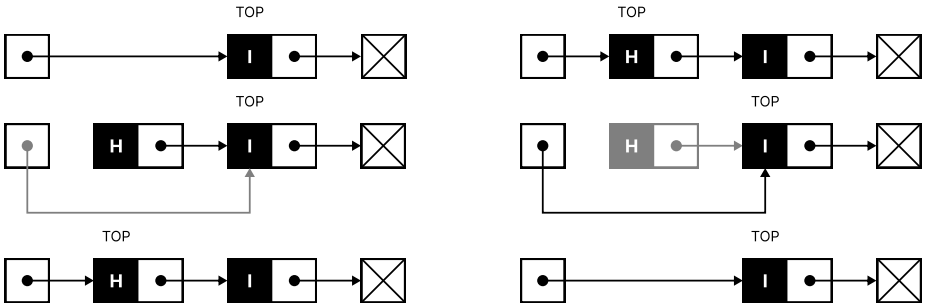

Fig. 2.4 Pushing element 'H' to the top of a linked list stack and creating 'HI' (left), and then removing it (right).

2.1.2 The prideful array

The linked list is kinda complicated, and compared to our initial book analogy there's a better way to represent stacks - arrays. Another problem is cache locality, which makes linked list stacks slower since elements may be stored huge memory distances away and loading them to faster, but smaller, memory areas for speed may be impossible. On the other hand, there are some improvements that can be made even with lists, but back to the topic.

An array is just a linear data structure made up of continuous memory. This simple concept makes it one of the most versatile data structures in all of programming.

Adding, removing and accessing the first top node only requires a numerical index value that can be inferred through the stack's length We just store the $LENGTH$ of the stack, then add an element by putting it into the $INDEX = LENGTH$ and incrementing $LENGTH$ by one; removing an element requires decrementing $LENGTH$ by one and making $INDEX = LENGTH$. Just don't forget to check if a stack's maximum length is less than maximum/capacity size during the push; and length not being zero while popping. Accessing the topmost element is just $INDEX = LENGTH - 1$, without the decrement (see Figure 2.5).

The biggest concern is that arrays aren't infinite and if we want to add an arbitrary number of elements we need to expand it - if we have space then we simply increase the capacity value, one must also store a changing capacity value for dynamically growing arrays and a constant maximum for static ones; else if the array occupies the maximum available chunk of memory all elements must be moved to a bigger chunk; and when that fails, because no chunk exists, everything's screwed and we must terminate.

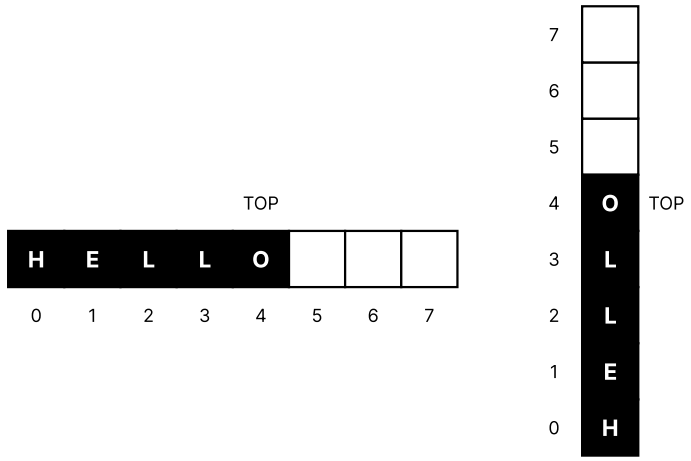

Fig. 2.5 A simple graphical array example. The top element is also the last element in the array.

2.1.3 Singly linked list of array elements

Combine the best of humbleness with pride and you get this. It's just a linked list, see Figure 2.6, but the node is actually made up of an array of elements instead of just a singleton. A mathematical person may say that the previous implementations are but a simple subset of this superset of a stack structure, but i digress.

The main benefit is that the list-array works like a array when accessing elements; and like a list when shortening/expanding it (we don't need to resize an array, only add a new node to the top). Adding and removing elements just combines the array method until full/empty, where it then performs the list operations. In hindsight, the implementation is more like a compromise than the best of both worlds.

To get the top element we can store the total element count as $LENGTH$, then simply getting the head node gives us access to the array with the top element so that lastly we only need to do $INDEX = (LENGTH - 1) \bmod CHUNK$, where $CHUNK$ is the node's array size.

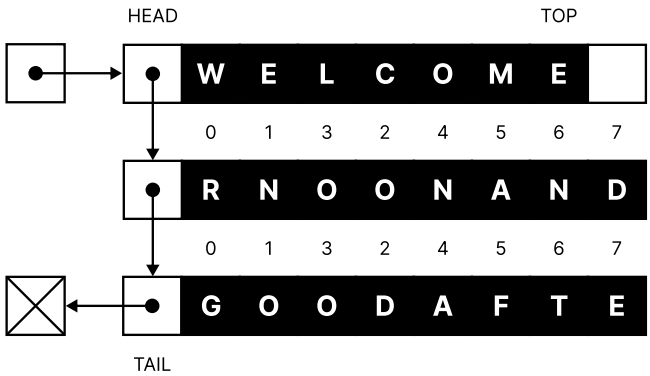

Fig. 2.6 Graphical example of singly linked list of arrays. The top element is the last element in the head node's array.

2.2 Array stack structure code

At the end I have decided to pick simple array-based stacks for one simple reason - continuous memory layout. I want to allow users to sort elements in the stack in order the have the same functionality as a priority queue, i.e. retrieving elements based on a priority, like the minimum. Of course the priority would be in reverse since the stack would get the last/top element in the array. It is possible to sort elements using the aforementioned implementations, but that requires creating a temporary array to copy elements back and forth. The next chapters of this part will use linked list restricted sequential

structures. In chapter 4 the double ended queue data structure, which is implemented like a list of arrays, can also be used as a stack alternative with the list-array advantages.

2.2.1 The infinite stack structure

The `istack` code uses a dynamic array of elements based on a user-defined size for a singular element, for example using `sizeof` on `int`. The three members `size`, `length` and `capacity` represent single element size, current stack length and maximum capacity before resizing is required. Lastly, the memory allocator allocates, reallocates and frees the elements array and other temporary structures used in certain functionalities.

include/sequence/istack.h

```
/// @brief Stack data structure.
typedef struct infinite_stack {
    char * elements;               // array of elements
    size_t size, length, capacity; // size of single element, structure
     length and its capacity
    memory_s const * allocator;
} istack_s;
```

2.2.2 Shrinking, expanding, resizing

Infinite expansion and shrinking require the understanding of reallocating memory and implementation of the `_istack_resize` function. It just calls standard `realloc` when high- and low-end capacity is reached while preserving minimum chunk length.

source/sequence/istack.c

```
/// @brief Resizes (reallocates) stack parameter arrays based on changed
     capacity.
/// @param stack Structure to resize.
/// @param size New size.
void _istack_resize(istack_s * const stack, size_t const size);
```

2.2.3 Creating the stack

The `create_istack` function creates an empty infinite stack structure storing the element size and allocator.

source/sequence/istack.c

```
/// @brief Creates an empty structure.
```

```
/// @param size Size of a single element.
/// @return Stack structure.
istack_s create_istack(size_t const size);
```

The creation process uses a compound literal to quickly create and return a new infinite stack structure. It only needs to set the single element `size` and `allocator` members.

source/sequence/istack.c

```
{
    // ...
    return (istack_s) { .size = size, .allocator = &standard };
}
```

2.2.4 The makings of a stack

The `make_istack` function, similar to `create_istack`, creates an infinite stack structure, but with the ability to specify a custom memory allocator.

source/sequence/istack.c

```
/// @brief Creates an empty structure.
/// @param size Size of a single element.
/// @param allocator Custom allocator structure.
/// @return Stack structure.
istack_s make_istack(size_t const size, memory_s const * const allocator);
```

The code is basically the same as the aforementioned function, but with the `standard` allocator replaced with the parameter one.

source/sequence/istack.c

```
{
    // ...
    return (istack_s) { .size = size, .allocator = allocator };
}
```

2.2.5 Stack destruction

Next, `destroy_istack` destroys the stack and all its elements while at the same time making it invalid for future usage. The parameters include a pointer to the stack and a function pointer taking in each element, allowing the user to manipulate elements prior to their destruction. The `destroy()` function pointer can primarily be used to free memory, like strings, from the heap.

include/sequence/istack.h

```
/// @brief Destroys a structure, and its elements and makes it unusable.
/// @param stack Structure to destroy.
/// @param destroy Function pointer to destroy a single element.
void destroy_istack(istack_s * const stack, set_fn const destroy);
```

Since our elements are stored in an array, i.e. continuous memory, it is possible to write a for loop with a pointer representing a single element to call the `destroy()` function pointer on. The variable then gets incremented by the specified size until the last valid memory address is reached.

source/sequence/istack.c

```
{
    // ...
    // iterate over each element and call destroy function on it
    for (char * e = stack->elements; e < stack->elements + (stack->length *
     stack->size); e += stack->size) {
        destroy(e);
    }
    // ...
```

After destroying each element the next step is to free the element's array using the specified `allocator` struct member.

source/sequence/istack.c

```
    // ...
    stack->allocator->free(stack->elements, stack->allocator->arguments);
    // ...
```

All that's left is to invalidate the stack, or set everything to zero, via calling `memset` from `<string.h>`.

source/sequence/istack.c

```
    // ...
    memset(stack, 0, sizeof(istack_s));
}
```

The `memset` function allows us to effectively set every member of a struct or any array to zero. It should only be used to set values to either all zero or all one (by using `-1` as the second parameter).

Listing 2.1 The `memset` function prototype.

```
#include <string.h>

void *memset(void s[n], int c, size_t n);
```

Linux manual page - memset(3)

The `memset()` function fills the first n bytes of the memory area pointed to by s with the constant byte c.

2.2.6 Clearing up the stack

Similar to `destroy_istack`, `clear_istack` can be used to destroy each element in our structure, but the stack remains valid for next use.

include/sequence/istack.h

```
/// @brief Clears a structure, and destroys its elements, but remains
    usable.
/// @param stack Structure to destroy.
/// @param destroy Function pointer to destroy a single element.
void clear_istack(istack_s * const stack, set_fn const destroy);
```

This is achieved by freeing the elements array and setting the stack's length and capacity to zero, and elements array NULL.

source/sequence/istack.c

```
{
    // ...
    // set length to zero
    stack->length = stack->capacity = 0;
    stack->elements = NULL;
}
```

2.2.7 The deep/shallow copy

Creating a copy can be done with the `copy_istack` function. This functionality is similar to the `memcpy` function in `<string.h>`, which takes a destination address, source address, and the size of the memory area to copy, see 2.2.

Listing 2.2 The `memcpy` function prototype from the Linux/UNIX manual page.

```
#include <string.h>

void *memcpy(void dest[restrict n], const void src[restrict n], size_t n);
```

Linux manual page - memcpy(3)

The `memcpy()` function copies n bytes from memory area `src` to memory area `dest`. The memory areas must not overlap. Use `memmove(3)` if the memory areas do overlap.

include/sequence/istack.h

```
/// @brief Creates a copy of a structure and all its elements.
/// @param stack Structure to copy.
/// @param copy Function pointer to create a deep/shallow copy of a single
    element.
/// @return Stack structure.
istack_s copy_istack(istack_s const * const stack, copy_fn const copy);
```

First and foremost, a replica stack structure is created and initialized, with all members except the elements array being directly copied. Then the original `allocator` is used to generate a memory array.

source/sequence/istack.c

```
{
    // create replica to initialize and return
    istack_s const replica = {
        .capacity = stack->capacity, .length = 0, .size = stack->size,
        .elements = stack->allocator->alloc(stack->capacity * stack->size,
    stack->allocator->arguments),
    };
```

All elements are later copied individually from source stack's memory at index position into replica's elements array.

source/sequence/istack.c

```
    // initialize replica's elements array with stack's elements
    for (size_t i = 0; i < stack->length; ++i) {
        size_t const offset = i * stack->size;
        copy(replica.elements + offset, stack->elements + offset);
    }

    return replica;
}
```

2.2.8 Checking emptiness

Since we can't remove elements from an empty stack, the function available to aid with this issue is the `is_empty_istack`.

include/sequence/istack.h

```
/// @brief Checks if structure is empty.
/// @param stack Structure to check.
/// @return 'true' if empty, 'false' if not.
bool is_empty_istack(istack_s const * const stack);
```

source/sequence/istack.c

```
{
    // ...
    return !(stack->length); // return negated length
}
```

2.2.9 Pushups to the top

To add an element one can use the the `push_istack` function. Since the stack is implemented using a resizable array checking if maximum capacity has been reached is a must. The index of where to push the element can be gained through the stack length member.

include/sequence/istack.h

```
/// @brief Pushes a single element to the top of the structure.
/// @param stack Structure to push into.
/// @param element Element buffer to push.
void push_istack(istack_s * const restrict stack, void const * const
    restrict element);
```

Firstly, checking if stack's length has reached the capacity for resizing into a new chunk of memory. Since the structures grow exponentially, starting at a specific power of two chunk, it is important to resize it into `ISTACK_CHUNK` if `length` member is zero and else to continue with doubling.

source/sequence/istack.c

```
{
    // ...
    if (stack->length == stack->capacity) { // if length is equal to
    capacity the array must expand linearly
        size_t const capacity = stack->length ? stack->length *
    CERPEC_FACTOR : ISTACK_CHUNK;
        _istack_resize(stack, capacity);
    }
```

As the size of a single element is user-define, the only optimal way to copy the element into our structure is to use `memcpy`. Because the `element` parameter and `elements` stack member are stored in two different memory locations `memcpy` and its speed can be utilized.

source/sequence/istack.c

```
    // ...
    // push element knowing the elements array can fit it
    memcpy(stack->elements + (stack->length * stack->size), element, stack
    ->size);
    stack->length++;
}
```

2.2.10 Popping it

Removal of elements is done through the `pop_istack` function. If the structure is empty the program logically errors.

include/sequence/istack.h

```
/// @brief Pops a single element from the top of the structure.
/// @param stack Structure to pop from.
/// @param buffer Element buffer to save pop.
void pop_istack(istack_s * const restrict stack, void * const restrict
    buffer);
```

Popping the top element is as simple as getting the element at decrementing `length`, using it as the index position and copying into a temporary variable parameter.

source/sequence/istack.c

```
{
    // ...
    // remove element from elements array
    stack->length--;
    memcpy(buffer, stack->elements + (stack->length * stack->size), stack->
    size);
```

The structure adjusts itself during growth, therefore it is important to not waste memory. Thus after reaching a smaller power of two capacity the stack automatically shrinks. Chunk length is a special case where if the capacity goes below it, the memory array stays the same, but only until reaching zero. After that it is set to zero.

source/sequence/istack.c

```
    // ...
    if (stack->length ≤ stack->capacity / CERPEC_FACTOR && (stack->length
    > ISTACK_CHUNK || !stack->length)) {
        _istack_resize(stack, stack->length);
    }
}
```

2.2.11 Reaching the peep

Peeping the last added element via `peep_istack` is a little bit like popping it.

include/sequence/istack.h

```
/// @brief Peeps a single element from the top of the structure.
/// @param stack Structure to peep.
/// @param buffer Element buffer to save peep.
void peep_istack(istack_s const * const restrict stack, void * const
    restrict buffer);
```

The idea is to just get the element index at $LENGTH - 1$ and save it into a temporary parameter.

source/sequence/istack.c

```
{
    // ...
    // only copy the top element into the buffer
    memcpy(buffer, stack->elements + ((stack->length - 1) * stack->size),
    stack->size);
}
```

2.2.12 For each element

Each element can be accessed and manipulated through the special `each_istack` function which iterates beginning not at the top, but at the bottom, and them making its way up to the top. The reason it starts at the bottom is because it makes printing the values easier.

include/sequence/istack.h

```
/// @brief Iterates over each element in structure starting from the
    beginning.
/// @param stack Structure to iterate over.
/// @param handle Function pointer to handle each element reference using
    generic arguments.
/// @param arguments Generic arguments to use in function pointer.
void each_istack(istack_s const * const restrict stack, handle_fn const
    handle, void * const restrict arguments);
```

All the element are in an array, thus iterating over them is as simple as using an empty for loop. The `handle` function pointer, together with `arguments`, allows direct access to the element, thus manipulation is possible. If `handle` returns false, the iteration automatically stops, kinda like the `break` keyword in a classic for loop.

source/sequence/istack.c

```
{
```

```
    // ...
    // iterate over each element from bottom to the top of stack
    for (char * e = stack->elements; e < stack->elements + (stack->length *
     stack->size) && handle(e, arguments); e += stack->size) {}
}
```

2.2.13 Applying mostly sorting

The `apply_istack` gives access to all the elements as if they were inside a continuous array, which is really simple for this stacks, but it is extremely hard to implement for structures in other chapters, like Queue and Deque. The main use of `apply_istack` is to sort the array of elements as fast as the sorting function pointer `process` and its generic `arguments` allow.

include/sequence/istack.h

```
/// @brief Apply each element in structure into an array to manage.
/// @param stack Structure to map.
/// @param process Function pointer to process array of elements using
    structure length and arguments.
/// @param arguments Generic arguments to use in function pointer.
void apply_istack(istack_s const * const restrict stack, process_fn const
    process, void * const restrict arguments);
```

Similarly to the `each_istack`, the `apply_istack` only has one line of code - the `process` function pointer itself.

source/sequence/istack.c

```
{
    // ...
    // process stack elements as an array (as a whole)
    process(stack->elements, stack->length, arguments);
}
```

2.3 Example usage

2.3.1 Creating and clearing/destroying a stack of integers

```
#include <sequence/istack.h>

void intdst(void * const element) {
    (*(int*)element) = 0; // to set to 0, or just use (void)(element);
}
```

```c
int main(void) {
    istack_s stack = create_istack(sizeof(int));

    // some operations

    clear_istack(&stack, intdst);

    // some new operations

    destroy_istack(&stack, intdst);

    return 0;
}
```

2.3.2 Pushing, peeping and popping integer elements

```c
#include <stdio.h>
#include <sequence/istack.h>

void intdst(void * const element) {
    (*(int*)element) = 0;
    // (void)(element);
}

int main(void) {
    istack_s stack = create_istack(sizeof(int));

    // push 10 elements
    for(int i = 0; i < 10; i++) {
        push_istack(&stack, &i);
    }

    // see if top element is 9
    int v = -1;
    peep_istack(&stack, &v);
    printf("%d\n", v);

    // pop and print all elements
    while(!is_empty_istack(&stack)) {
        pop_istack(&stack, &v);
        printf("%d ", v);
    }

    destroy_istack(&stack, intdst);

    return 0;
}
```

2.3.3 Iterating and sorting

```c
#include <stdio.h>
#include <stdlib.h>
#include <sequence/istack.h>

void intdst(void * const element) {
    (*(int*)element) = 0;
    // (void)(element);
}

int intrcmp(void * const a, void * const b) {
    return (*(int*)b)  - (*(int*)a); // reverse comparison
}

bool intprt(void * const element, void * format) {
    printf(format, (*(int*)element));
    return true;
}

void intsrt(void * const array, size_t const length, void * const compare)
    {
    qsort(array, length, sizeof(int), compare);
}

int main(void) {
    istack_s stack = create_istack(sizeof(int));

    // push 10 elements
    for(int i = 0; i < 10; i++) {
        push_istack(&stack, &i);
    }

    // print each element from bottom to top in order
    each_istack(&stack, intprt, "%d ");

    // sort elements in reverse
    // YOU SHOULD NOT CAST A FUNCTION POINTERS INTO VOID*
    // ARGUMENTS, THIS IS JUST A SILLY EXAMPLE, INSTEAD PUT
    // THE FUNCTION POINTER INSIDE A STRUCT AND PUT THE
    // INITIALIZED STRUCT POINTER ITSELF AS AN ARGUMENT
    apply_istack(&stack, intsrt, intrcmp);
    puts("");

    // print each element from bottom to top in reverse
    each_istack(&stack, intprt, "%d ");

    destroy_istack(&stack, intdst);

    return 0;
}
```

Chapter 3
Queue

Imagine you're waiting for your daily calorie intake, i.e. lunch, in school and you see a queue. When one person gets their lunch the entire queue moves one person to the front, because logically, the entire cafeteria won't move to the next person.

But that's not how queues in programming work, I mean, technically they can, but//... If each person is a single element and the cafeteria is at the current place of the first person in the queue, then you don't want to move each element to the cafeteria, but move the cafeteria itself to the next person. So we have a current pointer/index/position which, after getting rid of the first element, we go/increment/move to the second one, and so on.

Going back to chapter 2's chess analogy, before we put those moves into the stack we're kind of pulling them from an array of moves starting at the front until the back is reached. Now lets say there're move moves still happening, plus we want to play back and forth the moves which were already played without waiting for the game to finish. To solve this we can pipe the moves into a special queue as they happen, then push elements from the queue into the first stack. If we want to go to the beginning, then we push the moves from the first stack into the second. To play all the way to the latest move, all that's left is to first push all the elements from the second into the first stack, and push all moves from the queue into first. The current saved element is the latest move, see Figure 3.1.

3.1 Implementations of a queue

Similarly to stacks, there are also ~~three~~ two ways to implement queues:

- Singly linked lists
- Static arrays (not dynamic)

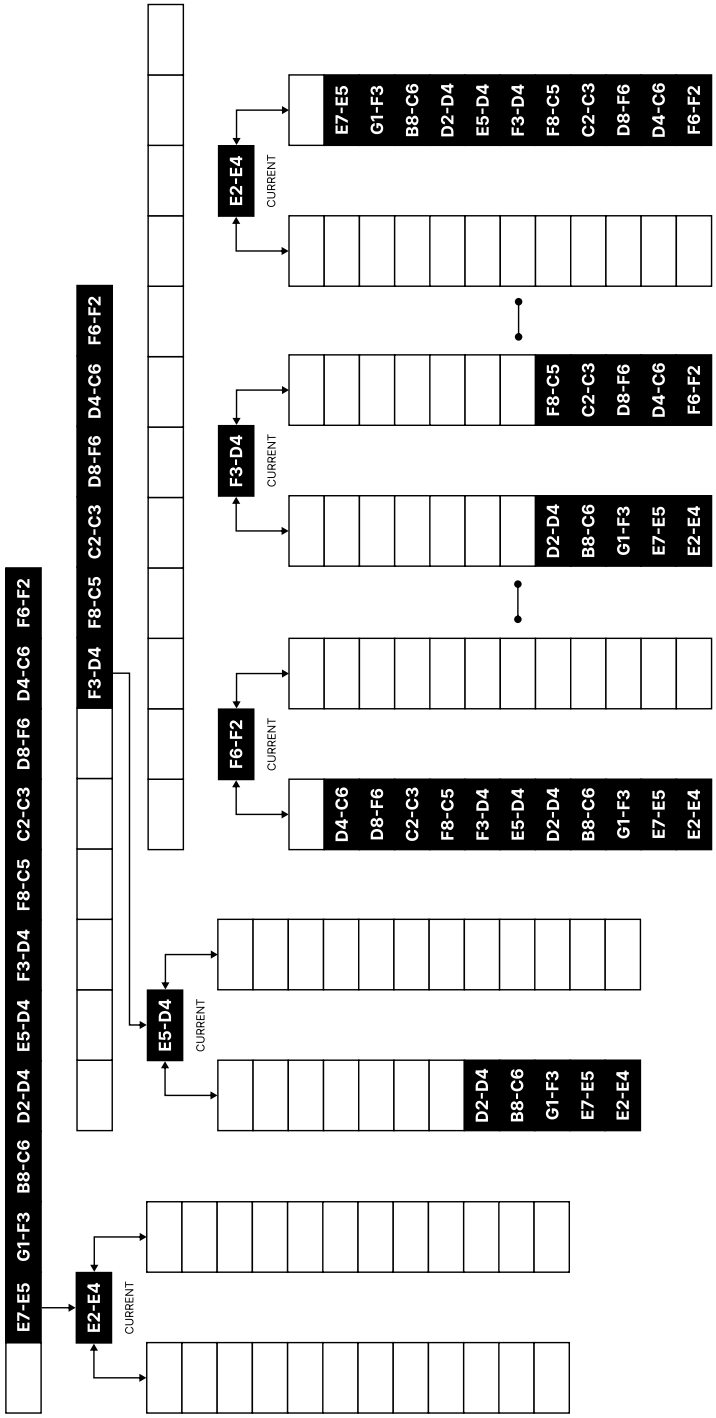

Fig. 3.1 Visual example of two stacks and a queue being used to track the chess game.

3.1.1 The free linked list

Usually when implementing a queue using linked lists we create a list structure that has one reference to the head - for access and removal, and to the tail node - for pushing elements.

However, there is also a slightly better way to implement them, which is to use circular lists. The next subsection talks about circular arrays, but we can also use circular lists, kinda like in Figure 3.2. Hear me out, this won't work if we only have a reference to the head, so lets go with the next best thing - the tail. Since the list is circular, tail's next node will therefore be the head, or front node.

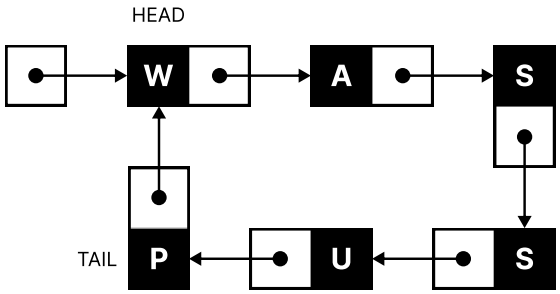

Fig. 3.2 Visual representation of a circular singly linked list.

This makes access and removal as simple as getting the head from the tail, also adding elements is easy since we just make the new node's next reference the head and change tail's next to new node, just see Figure 3.3. The downside is that circular linked lists are more complex than the classical list implementation, but by just having one tail node pointer we can decrease the size of the structure by quite a bit (or a whopping 32/64 bits at that).

3.1.2 The confined array

Another way to implement a queue is to use a static array. We can use dynamic arrays as well, but when the queue needs to expand the elements must be copied front to back into a new array. This sounds reasonable for array resizing, but usually when arrays resize they don't always need to copy all the elements and just either don't do anything or they just inform that the memory length was changed. This is the case for chapter 2's stacks, but in the case of queue a copy needs to happen all the time to move the elements to the beginning of the array, which is a huge performance hit. But what if we add the simplicity of a static array queue with the expandability of tail-based circular linked lists?

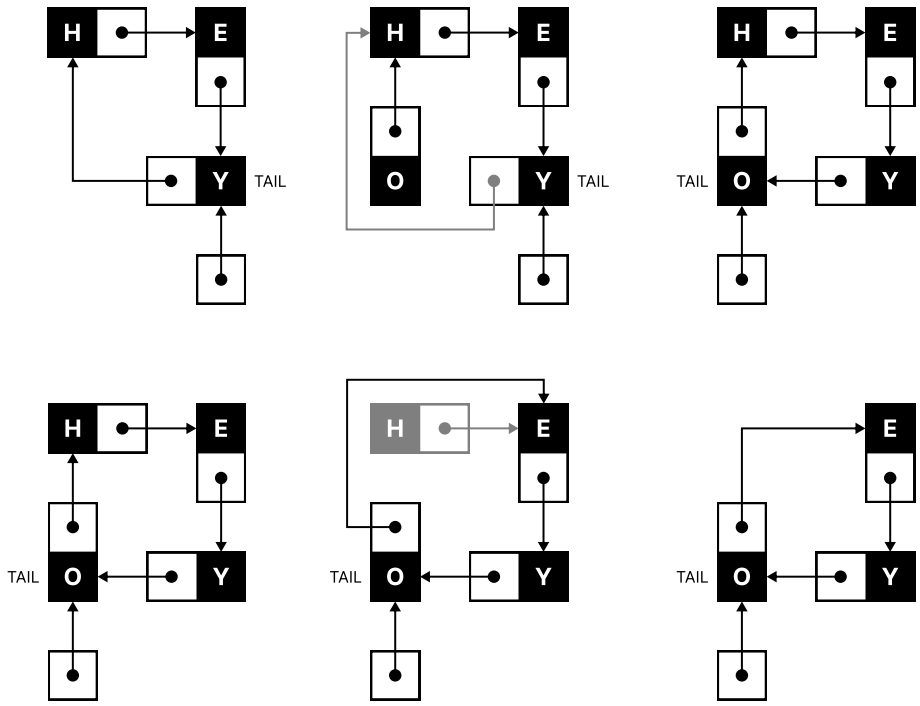

Fig. 3.3 Example of adding (top) and removing (bottom) an element from a circular singly linked list queue.

3.1.3 Tail-based circular linked list of arrays

So we have a tail-based circular linked list with nodes, but instead of the node containing only one element it has an entire array of elements. Technically, a simple linked list with one node element is a subset of all possible tail-based circular linked lists. So we have an array of continuous memory for each element, we get rid of the resizing and recopying from the start problem, and last, but least, kinda//... we may get better performance for big sizes since it's simpler to allocate a new node of size

$$136b = (8b + 128b) = (sizeof(next_pointer) + (32 * sizeof(integer_element)))$$

than it is to resize an elements array from 4MiB to 8MiB with potential copying. The downside is that the code complexity of tail-based circular linked lists and circular static arrays add up when combined together.

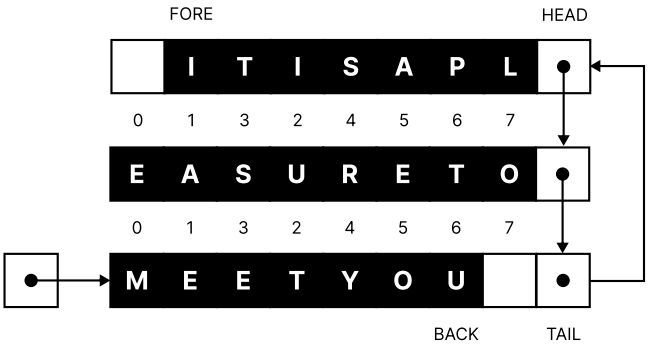

Fig. 3.4 Graphical example of tail-based circular linked list of arrays. The front/fore element is the first element (index one) in the head node's array, and the back element is the last element (index six) in the tail node's array.

3.2 Tail-based circular linked list of array code

Since I promised in chapter 2 to implement a linked list implementation of a restricted sequential data structure I've decided to do it with the queue (spoilers if you didn't read chapter 2: and also the Deque).

3.2.1 The infinite queue structure

The implementation is made up of two structures, lets call them the node and the queue.

The node uses a less known C feature - flexible array members, which are arrays without a specified size at the end of a structure - the structure is thus `pointer to next node` and `empty array member`.

The queue structure has a `tail` node pointer, the classic element `size` and structure `length`, and an `allocator` reference. It also includes a `current` index which we haven't talked about. The `current` index represents the position of the front element in head relative to the array, so when the index reaches the end it is set to zero. To get the back element we just add the `current` index with the queue `length` and multiply the result with the element `size`.

GNU C Language Manual - Flexible Array Fields

The C99 standard adopted a more complex equivalent of zero-length array fields. It's called a flexible array, and it's indicated by omitting the length, like this:

struct line { int length; char contents[]; };

The flexible array has to be the last field in the structure, and there must be other fields before it.

Under the C standard, a structure with a flexible array can't be part of another structure, and can't be an element of an array.

GNU C allows static initialization of flexible array fields. The effect is to "make the array long enough" for the initializer.

struct f1 { int x; int y[]; } f1 = { 1, { 2, 3, 4 } };

This defines a structure variable named f1 whose type is struct f1. In C, a variable name or function name never conflicts with a structure type tag.

Omitting the flexible array field's size lets the initializer determine it. This is allowed only when the flexible array is defined in the outermost structure and you declare a variable of that structure type. For example:

struct foo { int x; int y[]; }; struct bar { struct foo z; };

struct foo a = { 1, { 2, 3, 4 } }; // Valid.
struct bar b = { { 1, { 2, 3, 4 } } }; // Invalid.
struct bar c = { { 1, { } } }; // Valid.
struct foo d[1] = { { 1 { 2, 3, 4 } } }; // Invalid.

include/sequence/iqueue.h

```
/// @brief Circular linked list node for queue structure.
struct infinite_queue_node {
    struct infinite_queue_node * next; // next sibling node
    char elements[];                   // flexible elements array with
     IQUEUE_CHUNK length
};

/// @brief Infinite queue data structure.
typedef struct infinite_queue {
    struct infinite_queue_node * tail; // tail node to append next elements
      while enqueue-ing
    size_t size, current, length;      // current index, element size and
     structure length
    memory_s const * allocator;
} iqueue_s;
```

3.2.2 Creating the queue

The *create_iqueue* function create an empty circular list queue structure storing the allocator, index and size data.

source/sequence/iqueue.c

```
/// @brief Creates an empty structure.
/// @param size Size of a single element
/// @return Queue structure.
iqueue_s create_iqueue(size_t const size);
```

The creation process uses a compound literal to quickly create and return a new infinite queue structure. It only needs to set the single element `size` and `allocator` members.

source/sequence/iqueue.c

```
{
    // ...
    return (iqueue_s) { .size = size, .allocator = &standard };
}
```

3.2.3 The makings of a queue

The `make_iqueue` function, similar to `create_iqueue`, creates an infinite queue structure, but with the ability to specify a custom memory allocator.

source/sequence/iqueue.c

```
/// @brief Creates a custom empty structure.
/// @param size Size of a single element.
/// @param allocator Custom allocator structure.
/// @return Queue structure.
iqueue_s make_iqueue(size_t const size, memory_s const * const allocator);
```

The code is basically the same as the aforementioned function, but with the `standard` allocator replaced with the parameter one.

source/sequence/iqueue.c

```
{
    // ...
    return (iqueue_s) { .size = size, .allocator = allocator };
}
```

3.2.4 Queue destruction

Next up, the `destroy_iqueue` function destroys the queue structure and all of its nodes and elements. The element destruction is done by the `set_fn destroy` function pointer, which is called for each element before the node is freed. The node destruction is done by iterating through the circular list - not from the tail, but the head; and freeing each node until we get back to the tail. The code is a little bit more complex because of circularity and array node members.

include/sequence/iqueue.h

```
/// @brief Destroys a structure, and its elements and makes it unusable.
/// @param queue Structure to destroy.
/// @param destroy Function pointer to destroy a single element.
void destroy_iqueue(iqueue_s * const queue, set_fn const destroy);
```

Since our elements are stored in a linked list it is possible to iterate via going to the next node. The head node is the next node from the tail, so in order to begin destruction at the front we must reference the current node from the previous (tail) one.

source/sequence/iqueue.c

```
{
    // ...
    // for each node starting from tail (previous to head)
    struct infinite_queue_node * previous = queue->tail;
    for (size_t start = queue->current; queue->length; start = 0) {
        // ...
```

After that the array loop follows, which has a more complex logic, since the head and tail node can have different numbers of elements, unlike the in-between nodes which are always full. There can also be the case that only one node contains all elements, i.e. head equals tail, and it may not be full and the first and last few element slots may be empty.

source/sequence/iqueue.c

```
        // ...
        // destroy elements in previous' next (so head and next)
        size_t i = start;
        for (; i < queue->length && i < IQUEUE_CHUNK; ++i) {
            destroy(previous->next->elements + (i * queue->size));
        }
        queue->length -= (i - start);
        // ...
```

All that's left in the loop is to free the node and move to the next one until we get back to the tail, which is the last node to be freed.

source/sequence/iqueue.c

```
        // save destroyed node and cut it from queue list
        struct infinite_queue_node * temp = previous->next;
```

```
        previous->next = previous->next->next;

        // free destroyed node
        queue->allocator->free(temp, queue->allocator->arguments);
    }
```

Lastly we use `memset` to invalidate the queue from future use.

source/sequence/iqueue.c

```
    // set everything to zero
    memset(queue, 0, sizeof(iqueue_s));
}
```

3.2.5 Clearing up the queue

Similar to the destruction function, the `clear_iqueue` function clears the queue of all elements and nodes, but keeps the structure itself usable. The code is basically the same as the destruction function, but without the final `memset` and with only some members set to zero/NULL.

include/sequence/iqueue.h

```
/// @brief Clears a structure and destroys its elements, but remains usable
    .
/// @param queue Structure to destroy.
/// @param destroy Function pointer to destroy a single element.
void clear_iqueue(iqueue_s * const queue, set_fn const destroy);
```

source/sequence/iqueue.c

```
{
    // ...
    // for each node starting from tail (previous to head)
    struct infinite_queue_node * previous = queue->tail;
    for (size_t start = queue->current; queue->length; start = 0) {
        // destroy elements in previous' next (so head and next)
        size_t i = start;
        for (; i < queue->length && i < IQUEUE_CHUNK; ++i) {
            destroy(previous->next->elements + (i * queue->size));
        }
        queue->length -= (i - start);

        // save destroyed node and cut it from queue list
        struct infinite_queue_node * temp = previous->next;
        previous->next = previous->next->next;

        // free destroyed node
        queue->allocator->free(temp, queue->allocator->arguments);
    }
```

```
    queue->current = queue->length = 0;
    queue->tail = NULL;
}
```

3.2.6 The deep/shallow copy

Continuing with the high complexity, the next function is the `copy_iqueue` function, which creates a copy of the queue structure and all of its nodes and elements. The element copying is done by the `copy_fn copy` function pointer, which is called for each element to create either a deep or shallow copy.

include/sequence/iqueue.h

```
/// @brief Creates a copy of a structure and all its elements.
/// @param queue Structure to copy.
/// @param copy Function pointer to create a deep/shallow copy of a single
    element.
/// @return Queue structure.
iqueue_s copy_iqueue(iqueue_s const * const queue, copy_fn const copy);
```

First we initialize an empty queue replica.

source/sequence/iqueue.c

```
{
    // ...
    // create properly initialized replica
    iqueue_s replica = { .size = queue->size, .length = queue->length, .
    current = queue->current };
    // ...
```

Then we create a current pointer to queue's tail and a double pointer to replica's tail so as to be able to change replica's tail and its reference, and remove the special head case.

source/sequence/iqueue.c

```
    // ...
    // set original queue's and replica's current nodes for iteration
    struct infinite_queue_node const * current_queue = queue->tail;
    struct infinite_queue_node ** current_copy = &(replica.tail); // two
    pointer list to remove special .head case
    // ...
```

The loop starts with initializing the remaining elements count and starting index, like in `destroy_iqueue` and `clear_iqueue`. The loop ends when no elements to copy remain. The `last` pointer just contains the last created node to make the list circular and redirect the replica's tail to it since it would contain the head after the loop ends.

source/sequence/iqueue.c

```
    // ...
    struct infinite_queue_node * last = NULL;
    for (size_t remaining = queue->length, start = queue->current;
    remaining; start = 0) {
    // ...
```

The first part of the loop creates a new empty node node and redirects replica's and last's node to it.

source/sequence/iqueue.c

```
        // ...
        // allocate new node for replica
        last = (*current_copy) = queue->allocator->alloc(sizeof(struct
    infinite_queue_node) + (IQUEUE_CHUNK * queue->size), queue->allocator
    ->arguments);
        // ...
```

The second part just makes more redirects for replica's node and changes the queue's current node to the next.

source/sequence/iqueue.c

```
        // ...
        // redirect current replica's node with replica's tail for
    circularity
        (*current_copy)->next = replica.tail;
        current_queue = current_queue->next;
        // ...
```

The third part iterates over the node array and creates copies based on the copy function pointer. The logic is more complex since elements may not lay at the beginnings/ends of the array.

source/sequence/iqueue.c

```
        // ...
        // outside for loop to get copied chunk size, since it can either
    be 'remaining' or 'IQUEUE_CHUNK'
        size_t i = start;
        for (; i < remaining && i < IQUEUE_CHUNK; ++i) { // while i is less
     than both 'remaining' and 'IQUEUE_CHUNK'
            copy((*current_copy)->elements + (i * queue->size),
    current_queue->elements + (i * queue->size));
        }
        remaining -= i; // subtract copied size from remaining size using i
        // ...
```

Last part of the loop only redirects to the next replica node.

source/sequence/iqueue.c

```
        // ...
        current_copy = &((*current_copy)->next);
```

```
    }
    // ...
```

And with that the loop ends and all that's left is to properly redirect the replica's tail to the last node and return the initialized structure copy.

source/sequence/iqueue.c

```
    // ...
    replica.tail = last; // set replica's tail to its last added node (i.e.
     its tail)

    return replica;
}
```

3.2.7 Checking emptiness

Yep, the `is_empty_iqueue` function checks if the queue is empty by checking if the length is zero.

include/sequence/iqueue.h

```
/// @brief Checks if structure is empty.
/// @param queue Structure to check.
/// @return 'true' if empty, 'false' if not.
bool is_empty_iqueue(iqueue_s const * const queue);
```

source/sequence/iqueue.c

```
{
    // ...
    return (queue->length == 0);
}
```

3.2.8 Enqueueing it to the back

The `enqueue_iqueue` function adds an element to the back of the queue. The code is a little bit more complex because of the circular list and array members, but the logic is pretty simple - we check if the tail node is full, if it is we create a new one and redirect the tail to it, then we add the element to the back of the tail node's array and increment the length.

include/sequence/iqueue.h

```
/// @brief Enqueues a single element to the end of the structure.
/// @param queue Structure to enqueue into.
```

```
/// @param buffer Element buffer to enqueue.
void enqueue_iqueue(iqueue_s * const restrict queue, void const * const
    restrict buffer);
```

First we check if head's array is full, if yes then we must create a new node and perform pointer redirects, else we just copy the element to the node array.

source/sequence/iqueue.c

```
{
    // ...
    // index where the next element will be enqueued
    size_t const next_index = (queue->current + queue->length) %
    IQUEUE_CHUNK;
    if (!next_index) { // if head list array is full (is divisible) adds
    new list element to head
        // ...
```

In the condition we create a new node.

source/sequence/iqueue.c

```
        // ...
        struct infinite_queue_node * node = queue->allocator->alloc(sizeof(
    struct infinite_queue_node) + (IQUEUE_CHUNK * queue->size), queue->
    allocator->arguments);
        // ...
```

At the end of the condition we redirect the newly created node and queue's tail properly.

source/sequence/iqueue.c

```
        // ...
        if (queue->tail == NULL) {
            node->next = node; // create initial circle
        } else {
            node->next = queue->tail->next; // make temp's next node head
    node
            queue->tail->next = node; // make previous tail's next node
    point to temp
        }
        queue->tail = node;
    }
```

At last, we copy the element to its proper place in its proper node and increment the length.

source/sequence/iqueue.c

```
    // ...
    memcpy(queue->tail->elements + (next_index * queue->size), buffer,
    queue->size);
    queue->length++;
}
```

3.2.9 Dequeueing it

The `dequeue_iqueue` function removes the fist added element from the queue's list node at head.

include/sequence/iqueue.h

```
/// @brief Dequeues a single element from the start of the structure.
/// @param queue Structure to dequeue from.
/// @param buffer Element buffer to save dequeue.
void dequeue_iqueue(iqueue_s * const restrict queue, void * const restrict
    buffer);
```

Firstly the head node is reached and its first element is copied to the buffer, then queue's `length` is decremented and new `current` index is calculated.

source/sequence/iqueue.c

```
{
    // ...
    memcpy(buffer, queue->tail->next->elements + (queue->current * queue->
    size), queue->size);
    queue->length--; // decrement queue size
    queue->current = (queue->current + 1) % IQUEUE_CHUNK; // set current to
     next index in node array
```

If there are no more elements int the queue the last head/tail node is freed and everything is set to zero/NULL. Else if `current` has circled back to zero the current head node is empty and needs to be freed and removed. After that head's next node becomes the new head.

source/sequence/iqueue.c

```
    // ...
    if (!queue->length) { // if queue is empty after extracting element
    thne free memory and reset everything to zero
        queue->allocator->free(queue->tail, queue->allocator->arguments);
    // free empty tail/head node

        queue->current = 0; // if queue is empty make current index 0 to
    not break enqueue_iqueue operation
        queue->tail = NULL; // set tail to NULL
    } else if (queue->current == 0) { // else if current index circles back
    , free start list element and shift to next
        struct infinite_queue_node * head = queue->tail->next; // get empty
     head node
        queue->tail->next = queue->tail->next->next; // set new head node
    to its next node

        queue->allocator->free(head, queue->allocator->arguments); // free
    previous head node
    }
}
```

3.2.10 Peeking the front

`peek_iqueue` peek the front element at head and saves it into buffer without removing it from the queue.

include/sequence/iqueue.h

```
/// @brief Peeks a single element from the start of the structure.
/// @param queue Structure to peek.
/// @param buffer Element buffer to save peek.
void peek_iqueue(iqueue_s const * const restrict queue, void * const
    restrict buffer);
```

We just copy the `current` element at head into the buffer and return.

source/sequence/iqueue.c

```
{
    // ...
    memcpy(buffer, queue->tail->next->elements + (queue->current * queue->
    size), queue->size);
}
```

3.2.11 For each element

Each element can be accessed and manipulated through the special `each_iqueue` function which iterates from the front to the back, making its way backwards. That is standard queue traversal behavior.

include/sequence/iqueue.h

```
/// @brief Iterates over each element in structure starting from the
    beginning.
/// @param queue Structure to iterate over.
/// @param handle Function pointer to operate on each element reference
    using element size and generic arguments.
/// @param arguments Generic arguments to use in function pointer.
void each_iqueue(iqueue_s const * const restrict queue, handle_fn const
    handle, void * const restrict arguments);
```

Initially we need to save the remaining size as the queue's `length` and the previous node as the tail to start iteration from the head. Since head and tail may not have elements saved at the beginning/end of their arrays, we need to create the remaining size.

source/sequence/iqueue.c

```
{
    // ...
    size_t remaining = queue->length; // save queue size as remaining size
    for iteration
    struct infinite_queue_node const * previous = queue->tail;
```

The for loop iterates over each node and element until remaining size is zero. Since the first inserted element is stored in head the current node is inferred from the previous node's next reference. The secondary for loop just makes sure to begin iteration from the current index position in array and then continue from zero while elements are handled. The handle function is checked and ends the main `each_iqueue` function when handle returns false.

source/sequence/iqueue.c

```
    // ...
    // while remaining size wasn't decremented to zero
    for (size_t start = queue->current; remaining; start = 0, previous =
    previous->next) {
        struct infinite_queue_node * current = previous->next;

        size_t i = start; // save i outside loop to later use it in
    subtraction
        for (;i < remaining && i < IQUEUE_CHUNK; ++i) { // while i is less
    than either remaining size or node's array size
            // handle on element and if zero is returned then end main
    function
            if (!handle(current->elements + (i * queue->size), arguments))
    {
                return;
            }
        }
        remaining -= (i - start); // subtract absolute value of i and start
     from remaining size
    }
}
```

3.2.12 Apply sorting or something

`apply_iqueue` applies a `process` function pointer on an array of elements, thus allowing sorting of queue elements. It is way more complex than the `apply_istack` function from the previous chapter.

include/sequence/iqueue.h

```
/// @brief Apply each element in structure into an array to manage.
/// @param queue Structure to map.
/// @param process Function pointer to manage array of elements using
    strucuter length, element size and arguments.
/// @param arguments Generic arguments to use in function pointer.
void apply_iqueue(iqueue_s const * const restrict queue, process_fn const
    process, void * const restrict arguments);
```

At the beginning we allocate a temporary array to save the elements from the queue's nodes. The array `size` is the queue's `length` multiplied by the element size.

source/sequence/iqueue.c

```
{
    // ...
    // create elements array to temporary save element from circular linked
     list nodes
    char * elements_array = queue->allocator->alloc(queue->length * queue->
    size, queue->allocator->arguments);
```

Then we have our first evil for loop, which iterates over the nodes and elements in the same way as `each_iqueue`, but instead of handling each element it saves them into the temporary array. The logic is complex.

source/sequence/iqueue.c

```
    // ...
    size_t index = 0, remaining = queue->length; // temporary variable to
     save queue's size
    struct infinite_queue_node const * previous = queue->tail; // create
     node pointer to save previous nodes into
    for (size_t start = queue->current; remaining; start = 0, previous =
     previous->next) {
        size_t i = start; // extract i from for loop to later subtract it
     from remaining size
        for (;i < remaining && i < IQUEUE_CHUNK; ++i) { // while i is less
     than remaining size and node array size
            // save previous' next elements (so current) to elements array
     at index, and increment index
            memcpy(elements_array + (index * queue->size), previous->next->
     elements + (i * queue->size), queue->size);
            index++;
        }
        remaining -= (i - start); // subtract absolute value of i and start
      from remaining size
    }
    // ...
```

Now we can finally process the elements as a single array.

source/sequence/iqueue.c

```
    // ...
    process(elements_array, queue->length, arguments); // process
     initialized elements array
    // ...
```

Now we go the the second evil for loop, which basically just copies the elements from the array into the queue's nodes, thus saving the changes made by the process function. The logic is basically the same as the first loop, but instead of saving elements into the array it saves them back into the nodes.

source/sequence/iqueue.c

```
    // ...
    index = 0, remaining = queue->length, previous = queue->tail;
```

```
    for (size_t start = queue->current; remaining; start = 0, previous =
    previous->next) { // save elements array back into queue
        size_t i = start; // extract i from for loop to later subtract it
    from remaining size
        for (;i < remaining && i < IQUEUE_CHUNK; ++i) { // while i is less
    than remaining size and node array size
            memcpy(previous->next->elements + (i * queue->size),
    elements_array + (index * queue->size), queue->size);
            index++;
        }
        remaining -= (i - start); // subtract absolute value of i and start
     from remaining size
    }
    // ...
```

And all that's left is to free the temporary array and return.

source/sequence/iqueue.c

```
    // ...
    queue->allocator->free(elements_array, queue->allocator->arguments);
}
```

3.3 Example usage

3.3.1 Creating and clearing/destroying a queue of integers

```
#include <sequence/iqueue.h>

void intdst(void * const element) {
    (*(int*)element) = 0; // to set to 0, or just use (void)(element);
}

int main(void) {
    iqueue_s queue = create_iqueue(sizeof(int));

    // some operations

    clear_iqueue(&queue, intdst);

    // some new operations

    destroy_iqueue(&queue, intdst);

    return 0;
}
```

3.3.2 Enqueueing, peeking and dequeueing integer elements

```
#include <stdio.h>
#include <sequence/iqueue.h>

void intdst(void * const element) {
    (*(int*)element) = 0;
    // (void)(element);
}

int main(void) {
    iqueue_s queue = create_iqueue(sizeof(int));

    // enqueue 10 elements
    for(int i = 0; i < 10; i++) {
        enqueue_iqueue(&queue, &i);
    }

    // see if front element is 0
    int v = -1;
    peek_iqueue(&queue, &v);
    printf("%d\n", v);

    // dequeue and print all elements
    while(!is_empty_iqueue(&queue)) {
        dequeue_iqueue(&queue, &v);
        printf("%d ", v);
    }

    destroy_iqueue(&queue, intdst);

    return 0;
}
```

3.3.3 Iterating and sorting

```
#include <stdio.h>
#include <stdlib.h>
#include <sequence/iqueue.h>

void intdst(void * const element) {
    (*(int*)element) = 0;
    // (void)(element);
}

int intrcmp(void * const a, void * const b) {
    return (*(int*)b)  - (*(int*)a); // reverse comparison
}
```

```c
bool intprt(void * const element, void * format) {
    printf(format, (*(int*)element));
    return true;
}

void intsrt(void * const array, size_t const length, void * const compare)
    {
    qsort(array, length, sizeof(int), compare);
}

int main(void) {
    iqueue_s queue = create_iqueue(sizeof(int));

    // enqueue 10 elements
    for(int i = 0; i < 10; i++) {
        enqueue_iqueue(&queue, &i);
    }

    // print each element from start to end in order
    each_iqueue(&queue, intprt, "%d ");

    // sort elements in reverse
    // YOU SHOULD NOT CAST A FUNCTION POINTERS INTO VOID*
    // ARGUMENTS, THIS IS JUST A SILLY EXAMPLE, INSTEAD PUT
    // THE FUNCTION POINTER INSIDE A STRUCT AND PUT THE
    // INITIALIZED STRUCT POINTER ITSELF AS AN ARGUMENT
    apply_iqueue(&queue, intsrt, intrcmp);
    puts("");

    // print each element from start to end in reverse
    each_iqueue(&queue, intprt, "%d ");

    destroy_iqueue(&queue, intdst);

    return 0;
}
```

Further reading

[1] cppreference.com. std::queue. https://en.cppreference.com/cpp/container/queue. [Online; accessed 20-April-2026]. 2026.

[2] Oracle. Queue (Java Platform SE 8). https://docs.oracle.com/javase/8/docs/api/java/util/Queue.html. [Online; accessed 20-April-2026]. 2014.

[3] Mitchell Dalvi Rosen and Travis Staton. Queue. https://hackage.haskell.org/package/queues-1.0.0/docs/Queue.html. Accessed 20-Apr-2026. 2024.

[4] W3Schools. DSA Queues. https://www.w3schools.com/dsa/dsa_data_queues.php. [Online; accessed 20-April-2026]. 2026.

Chapter 4
Deque

Part II

Linked lists

Lorem ipsum.